Differential Expression

> An answer `md` file for Bioinformatics_Homework_RNAseq_Differential_Expression

> Direct to T1, T2, T3, T4, T5 quickly here.

T1

> Multiple test correction

> Direct to top quickly here.

1. Multiple test correction

- Definition
  - A statistical **adjustment** when conducting **multiple hypothesis tests simultaneously**
  - Addresses inflated error rates (**false positives**) due to **repeated** testing
- Example
  - Testing `1000` genes with a `p-value` threshold of `0.05` yields about `50` false positives *by chance*
  - Bengamini-Hochberg (`FDR` control) is a common method for multiple test correction

2. Differences between `p-` and `q-value`

2.1 `p-value`

- Is the **probability** of observing data **as actual results** under the assumption that **null hypothesis is true**
- Controls **per-comparison** error rate
  - If null is true, there's a `5%` chance of seeing this result.

2.2 `q-value` , `FDR`

> `FDR` , False Discovery Rate

- Is the **expected proportion of false positives** among **all** rejected hypotheses
- Controls false discovery rate across **multiple tests**
  - There's an expected `5%` of significant yields that are false positives.

- Formula

$$q_{(i)} = \min_{k \geq i} \left(\frac{m \times p_{(k)}}{k} \right)$$

2.3 Key differences

Value	Scope	Interpretation
`p-value`	Single test	This result has a `5%` chance to occur assuming null is true.
`q-value`	Multiple test	This set of significant results contains about `5%` expected false positives.

T2

> Normalization methods in `edgeR` and `DESeq2`

> Direct to top quickly here.

1. Normalization in `edgeR`：`TMM`

> `TMM` , Trimmed Mean of `M-values`

- Basic assumption
  - Most genes are **not differentially expressed** genes
  - **Normalization factor** could be calculated from weighted average of `M-values` to counterbalance deviations brought by **sequencing depth**
- Calculation method
  - a. **Select a reference sample**
    - Select the sample of which the `75%` **quantile** of all gene expression levels is the **closest to average**
    - This sample is **reference sample**, marked as r
  - b. **Calculate M_g and A_g values for each gene g**

    > K_{gj} = raw count of gene g in sample j
    > D_j = all raw counts in sample j

    - `M-value` indicates **fold change** of gene g in sample j relative to r

$$M_g\left(j, r\right) = \log_2 \frac{K_{gj}/D_j}{K_{gr}/D_r}$$

- `A-value` indicates the expression of gene g in sample j relative to r

$$A_g(j,r) = \frac{1}{2}\log_2\left(\frac{K_{gj}}{D_j} \times \frac{K_{gr}}{D_r}\right)$$

- c. **Select a representative gene set**
  - Ensures a reliable gene set for calculating **normalization factor** by excluding potential **differentially expressed** genes
  - Trim `M-value` by `30%` and trim `A-value` by `5%`
  - The representative gene set after trimming is marked as G
- d. **Calculate weight for each gene in G**
  - Bestow higher weight upon genes with higher counts and higher stability

$$w_g(j,r) = \left(\frac{D_j - K_{gj}}{D_j \times K_{gj}} + \frac{D_r - K_{gr}}{D_r \times K_{gr}}\right)^{-1}$$

- e. **Calculate `TMM` value for sample j**
  - `TMM` value is practically a weighted average of M_g

$$\mathrm{TMM}(j,r) = \frac{\sum_{g\in G} w_g(j,r) \times M_g(j,r)}{\sum_{g\in G} w_g(j,r)}$$

  - It represents the **deviation** of sample j from r
- f. **Apply ultimate normalization factor**
  - Calculate **normalization factor** C_j

$$C_j = 2^{\mathrm{TMM}(j,r)}$$

  - Apply C_j to **normalize** counts

$$(\text{Normalized count})_{gj} = \frac{K_{gj}}{C_j}$$

2. Normalization in `DESeq2`：`RLE`

> `RLE` , Relative Log Expression

- Basic assumption
  - Create a **virtual reference** sample using geometric mean
  - Calculate the **size factor** of each sample relative to this reference sample
- Calculation method
  - a. **Create virtual reference**

> N = total sample count
> K_{gj} = raw count of gene g in sample j

- Calculate geometric mean for each gene g

$$\mu_g = \sqrt[N]{\prod_{j=1}^{N} K_{gj}}$$

- b. **Calculate initial ratio matrix**
  - Calculate ratio

$$\mathrm{Ratio}_{gj} = \frac{K_{gj}}{\mu_g}$$

  - If $\mathrm{Ratio}_{gj} > 1$, the expression of gene g in sample j is higher than average and *vice versa*
- c. **Calculate size factor for each sample**
  - **Median** of ratios represents the deviation of sample j from r

$$s_j = \mathrm{median}_g \mathrm{Ratio}_{gj}$$

  - **Median** is better than **mean** because **mean** is greatly affected by **extreme values**
- d. **Normalize counts with size factors**

$$(\mathrm{Normalized\ count})_{gj} = \frac{K_{gj}}{s_j}$$

T3

> Finding differentially expressed genes in *uvr8* using `DESeq2` and `edgeR`

> Direct to top quickly here.

1. `R` Script

- You may download `R` script for both `DESeq2` and `edgeR` here
- The scripts are also shown here (direct to `DESeq2` or `edgeR`)
  > Click here to skip the following codes

Script for `DESeq2`

```r
# Read from table and filter uvr8 counts
raw.counts <- read.table(
    "count_exon.txt",
    sep='\t',
    header = TRUE,
    row.names = 1)

uvr8.raw.counts <- raw.counts[,c(
    "UD1_1", "UD1_2", "UD1_3",
    "UD0_1", "UD0_2", "UD0_3")]

uvr8.filtered.counts <- uvr8.raw.counts[rowMeans(
    uvr8.raw.counts) > 5, ]

# Generate condition and colData for DESeq2
uvr8.conditions <- factor(
    c(rep("Control", 3), rep("Treatment", 3)),
    levels = c("Control","Treatment"))

uvr8.colData <- data.frame(
    row.names = colnames(uvr8.filtered.counts),
    conditions=uvr8.conditions)

# Use DESeq2 to analyze data
suppressPackageStartupMessages(library(DESeq2))
# Create dataset
uvr8.dataset <- DESeqDataSetFromMatrix(
    uvr8.filtered.counts,
    uvr8.colData,
    design = ~ conditions)
# Analyze
uvr8.dataset <- DESeq(uvr8.dataset)
uvr8.result <- results(uvr8.dataset)

# Output as .txt file
write.table(
    uvr8.result,
    "uvr8.light.vs.dark.all.DESeq2.txt",
    sep='\t',
    row.names = TRUE,
    quote = FALSE)
```

Script for `edgeR`

```
# Read from table and filter uvr8 counts
raw.counts <- read.table(
    "count_exon.txt",
    sep='\t',
    header = TRUE,
    row.names = 1)

uvr8.raw.counts <- raw.counts[,c(
    "UD1_1", "UD1_2", "UD1_3",
    "UD0_1", "UD0_2", "UD0_3")]

uvr8.filtered.counts <- uvr8.raw.counts[rowMeans(
    uvr8.raw.counts) > 5,]

# Generate condition and design for DESeq2
uvr8.conditions <- factor(
    c(rep("Control", 3), rep("Treatment", 3)),
    levels = c("Control","Treatment"))

uvr8.design <- model.matrix(~ uvr8.conditions)

# Use edgeR to analyze data
library(edgeR)
uvr8.result <- DGEList(counts = uvr8.filtered.counts)
# Generate TMM matrix
uvr8.result <- calcNormFactors(uvr8.result, method = "TMM")
# Esimate dispersion
uvr8.result <- estimateDisp(uvr8.result, design = uvr8.design)
fit <- glmFit(uvr8.result, design = uvr8.design)
# Likelihood ratio test (lrt)
lrt <- glmLRT(fit, coef = 2)

# Output as .txt file
uvr8.result <- topTags(lrt, n = nrow(uvr8.result))$table
write.table(
    uvr8.result,
    "uvr8.light.vs.dark.all.edgeR.txt",
    sep='\t',
    row.names = TRUE,
    col.names = TRUE,
    quote = FALSE)
```

2. `txt` Results

- You may download `txt` result files for both `DESeq2` and `edgeR` here

T4

> *Venn* diagram for comparison between `DESeq2` and `edgeR` results

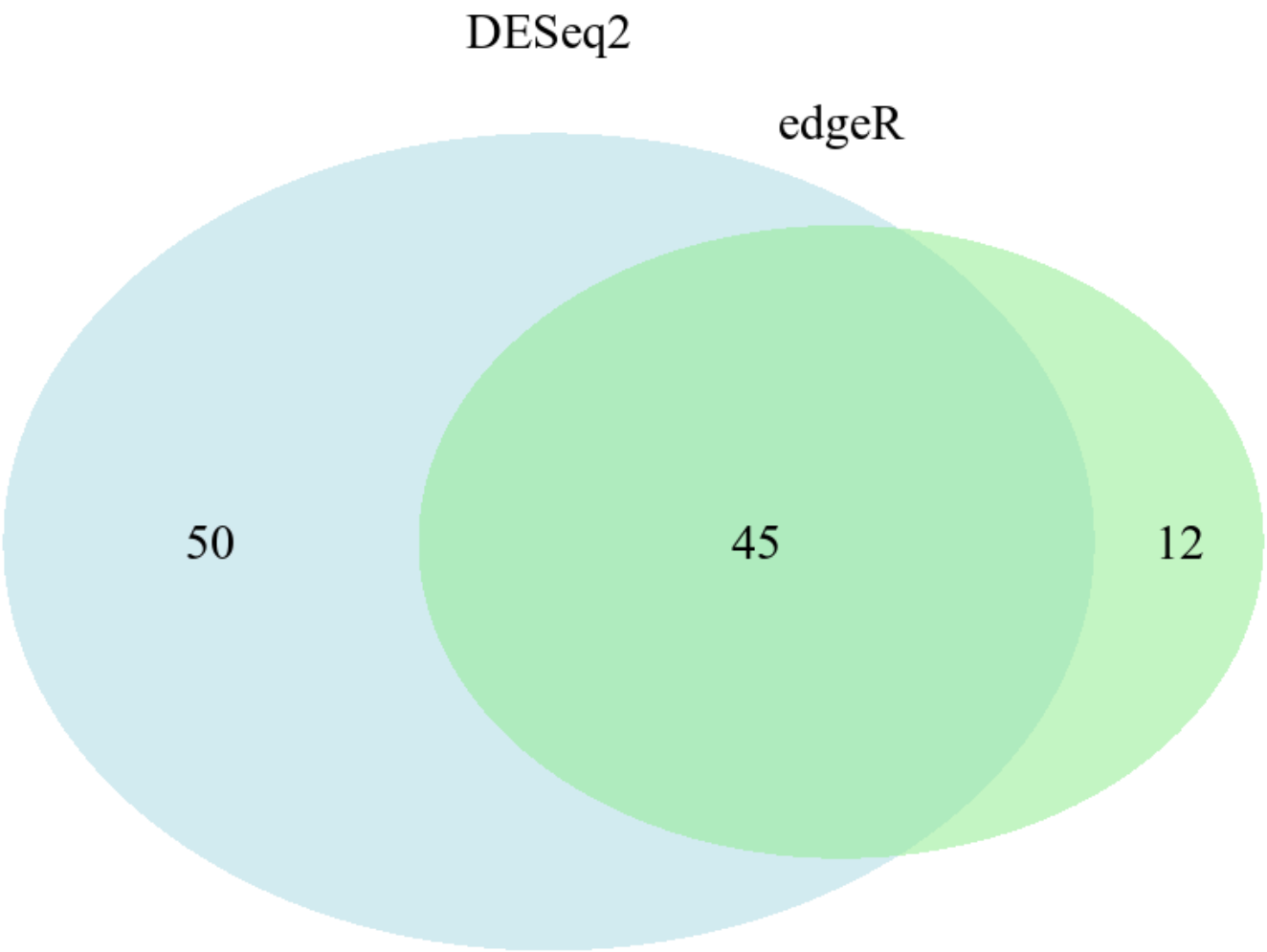
> Direct to top quickly here.

- For `R` script, download or view script here
  > Click here to skip the following code

```r
# Read from table
DESeq2.raw.data <- read.table(
    "uvr8.light.vs.dark.all.DESeq2.txt",
    header = TRUE, sep = "\t", row.names = 1)
edgeR.raw.data <- read.table(
    "uvr8.light.vs.dark.all.edgeR.txt",
    header = TRUE, sep = "\t", row.names = 1)

# Select genes with significant expression changes
DESeq2.filtered.data <- DESeq2.raw.data[
    !is.na(DESeq2.raw.data$padj) &
    abs(DESeq2.raw.data$log2FoldChange) > 1 &
    DESeq2.raw.data$padj < 0.05,]
edgeR.filtered.data <- edgeR.raw.data[
    abs(edgeR.raw.data$logFC) > 1 &
    edgeR.raw.data$FDR < 0.05,]

# Determine intersection number
DESeq2.genes <- rownames(DESeq2.filtered.data)
edgeR.genes <- rownames(edgeR.filtered.data)

intersect <- length(intersect(DESeq2.genes, edgeR.genes))
DESeq2.only <- length(setdiff(DESeq2.genes, edgeR.genes))
edgeR.only <- length(setdiff(edgeR.genes, DESeq2.genes))

# Create Venn diagram
library(VennDiagram)
grid.newpage()
venn.plot <- draw.pairwise.venn(
    area1 = DESeq2.only + intersect,
    area2 = edgeR.only + intersect,
    cross.area = intersect,
    category = c("DESeq2", "edgeR"),
    fill = c("lightblue", "lightgreen"),
    alpha = 0.5,
    lty = "blank",
    cex = 1.5,
    cat.cex = 1.5,
    cat.pos = c(0, 0),
    cat.dist = 0.05
)

# Output as png
```

```
png("T4_results_venndiagram.png", width = 800, height = 600, res = 120)
grid.draw(venn.plot)
dev.off()
```

- For *venn* diagram, download here



T5

> Selecting appointed genes and creating heatmap

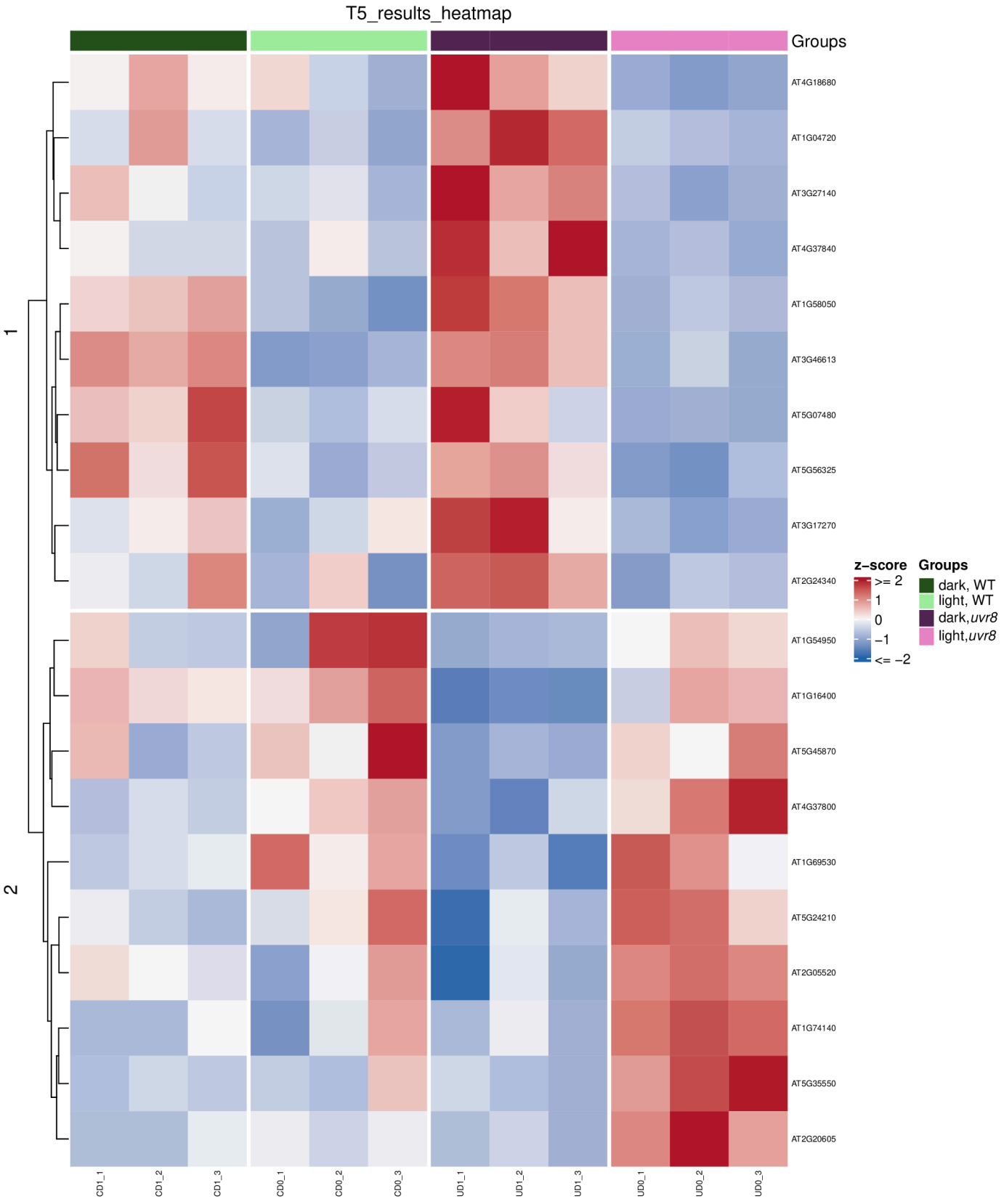
> Direct to top quickly here.

- For `R` script, download or view script here
  > Click here to skip the following code

```r
# Read from table
edgeR.raw.data <- read.table(
    "uvr8.light.vs.dark.all.edgeR.txt",
    header = TRUE, sep = "\t", row.names = 1)
raw.data <- read.table(
    "count_exon.txt",
    header = TRUE, sep = "\t", row.names =1)

# Select genes in edgeR with FDR < 0.05
edgeR.filtered.data <- edgeR.raw.data[
    edgeR.raw.data$FDR < 0.05,]
# Sort genes based on logFC
edgeR.sorted.data <- edgeR.filtered.data[
    order(
        edgeR.filtered.data$logFC,
        decreasing = TRUE
    ),]
num <- nrow(edgeR.sorted.data)
edgeR.selected.genes <- rownames(edgeR.sorted.data[
    c(1:10, (num - 9):num),])

# Calculate log10CPM and z-scores for raw.data
library(edgeR)
y <- DGEList(counts = raw.data)
CPM.matrix <- edgeR::cpm(y, log = F)
log10.CPM.matrix <- log10(CPM.matrix + 1)
z.scores.matrix <- (log10.CPM.matrix - rowMeans(log10.CPM.matrix))/apply(log10.CPM.matrix,

# Select previous genes
z.scores.matrix <- z.scores.matrix[
    rownames(z.scores.matrix) %in% edgeR.selected.genes,]

# Draw heatmap
suppressPackageStartupMessages(library(ComplexHeatmap))
suppressPackageStartupMessages(library(circlize))
# Annotations
ann <- HeatmapAnnotation(
    Groups = rep(c(
        "dark, WT", "light, WT", "dark, uvr8", "light, uvr8"
        ), each = 3),
    col = list(Groups = c(
        "dark, WT" = "#245219",
        "light, WT" = "#9deb9d",
```

```
        "dark, uvr8" = "#542354",
        "light, uvr8" = "#ea86c7"
    )),
    annotation_legend_param = list(
        Groups = list(
            title = "Groups",
            labels = c(
                "dark, WT",
                "light, WT",
                expression(paste("dark,", italic("uvr8"))),
                expression(paste("light,", italic("uvr8")))
            ),
            at = c("dark, WT", "light, WT", "dark, uvr8", "light, uvr8")
        )
    )
)

# Create map
heatmap <- Heatmap(
    z.scores.matrix,
    name = "T5_result_heatmap",
    column_title = "T5_results_heatmap",
    top_annotation = ann,
    cluster_columns = FALSE,
    column_split = rep(1:4, each = 3),
    row_split = 2,
    show_row_names = TRUE,
    show_column_names = TRUE,
    col = colorRamp2(c(-2, 0, 2), c("#2166AC", "#F7F7F7", "#B2182B")),  # 颜色设置
    row_names_gp = gpar(fontsize = 6),
    column_names_gp = gpar(fontsize = 6),
    heatmap_legend_param = list(
        title = "z-score",
        at = c(-2, -1, 0, 1, 2),
        labels = c("<= -2", "-1", "0", "1", ">= 2")
    )
)

# Save as pdf
pdf("T5_results_heatmap.pdf", width = 10, height = 12)
draw(heatmap)
dev.off()
```

- For heatmap, download in `png` or `pdf`


T5_results_heatmap
Groups
AT4G18680
AT1G04720
AT3G27140
AT4G37840
AT1G58050
AT3G46613
AT5G07480
AT5G56325
AT3G17270
AT2G24340
AT1G54950
AT1G16400
AT5G45870
AT4G37800
AT1G69530
AT5G24210
AT2G05520
AT1G74140
AT5G35550
AT2G20605
1
2
CD1_1
CD1_2
CD1_3
CD0_1
CD0_2
CD0_3
UD1_1
UD1_2
UD1_3
UD0_1
UD0_2
UD0_3
z−score
>= 2
1
0
−1
<= −2
Groups
dark, WT
light, WT
dark,uvr8
light,uvr8